

SECURE CODING REVIEW

Submitted By: Parimi Pujitha

Internship: CodeAlpha

Programming Language: Python

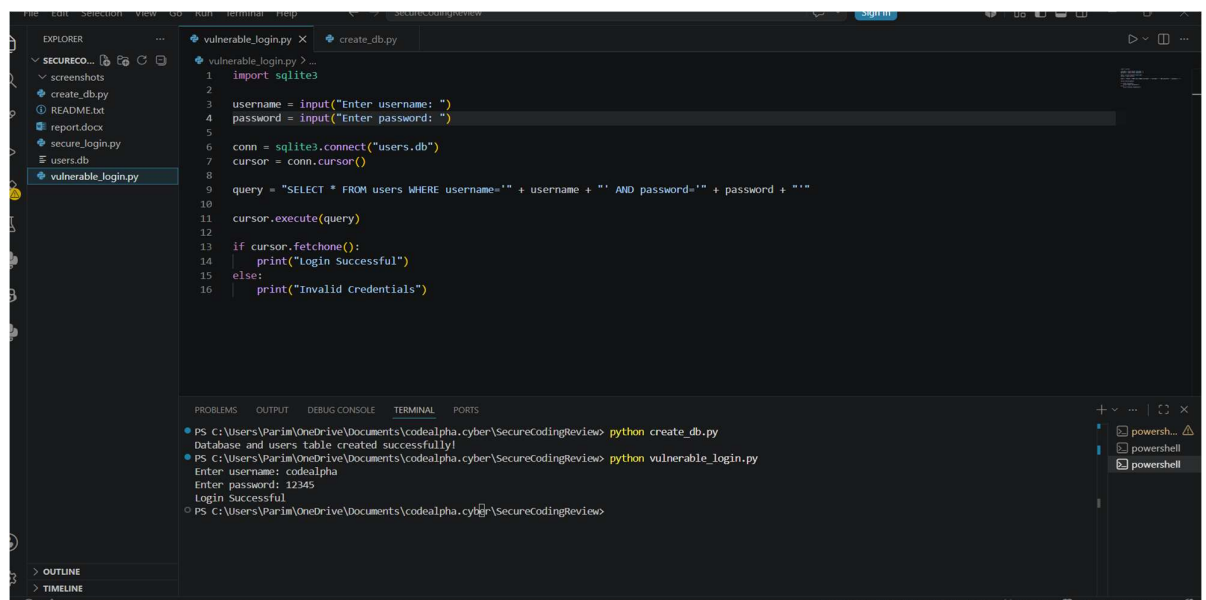
Project: Secure Login System Review

1. Introduction

Secure coding is the practice of developing software in a secure manner to reduce vulnerabilities and protect applications from cyber attacks. In this project, a Python login system was reviewed to identify common security issues and implement secure coding techniques.

2. Vulnerable Code Review

The vulnerable login system directly inserted user input into SQL queries. This created security risks such as SQL Injection attacks and insecure password handling.



The screenshot displays a code editor with a file explorer on the left showing a project named 'SECURECODINGREVIEW'. The main editor window shows the file 'vulnerable_login.py' with the following Python code:

```
1 import sqlite3
2
3 username = input("Enter username: ")
4 password = input("Enter password: ")
5
6 conn = sqlite3.connect("users.db")
7 cursor = conn.cursor()
8
9 query = "SELECT * FROM users WHERE username='" + username + "' AND password='" + password + "'"
10
11 cursor.execute(query)
12
13 if cursor.fetchone():
14     print("Login Successful")
15 else:
16     print("Invalid Credentials")
```

Below the code editor, the 'TERMINAL' tab shows the execution of two Python scripts:

```
PS C:\Users\Parim\OneDrive\Documents\codealpha.cyber\SecureCodingReview> python create_db.py
Database and users table created successfully!
PS C:\Users\Parim\OneDrive\Documents\codealpha.cyber\SecureCodingReview> python vulnerable_login.py
Enter username: codealpha
Enter password: 12345
Login Successful
PS C:\Users\Parim\OneDrive\Documents\codealpha.cyber\SecureCodingReview>
```

3. Vulnerabilities Identified

1. SQL Injection:

The application directly concatenated user input into SQL queries, allowing attackers to manipulate database operations.

2. Plain Text Password Storage:

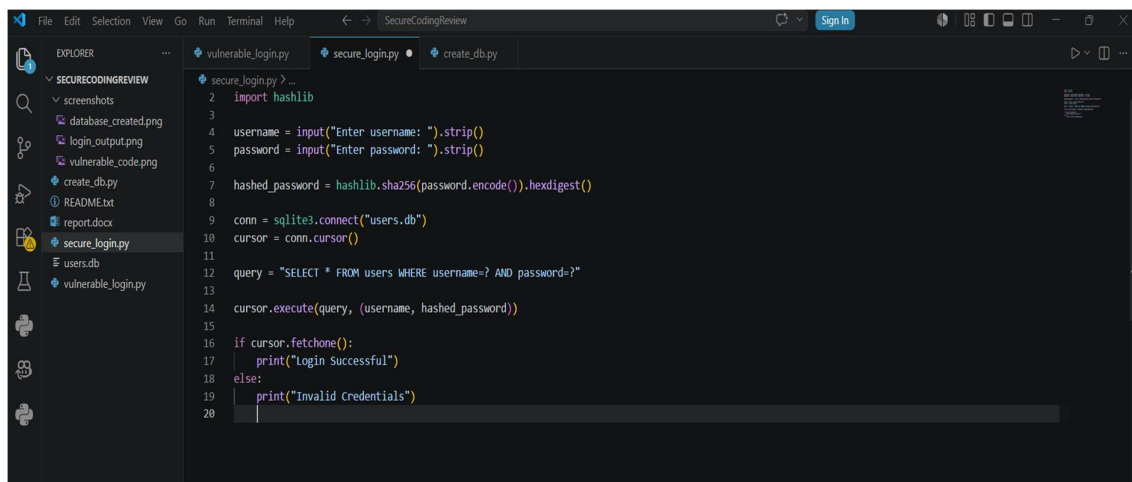
Passwords were stored without encryption, making them vulnerable if the database was leaked.

3. Lack of Input Validation:

The application accepted all input without proper validation or sanitization.

3. Secure Code Implementation

The secure version of the application used parameterized queries and password hashing to improve security and prevent common cyber attacks.



```
1 import sqlite3
2 import hashlib
3
4 username = input("Enter username: ").strip()
5 password = input("Enter password: ").strip()
6
7 hashed_password = hashlib.sha256(password.encode()).hexdigest()
8
9 conn = sqlite3.connect("users.db")
10 cursor = conn.cursor()
11
12 query = "SELECT * FROM users WHERE username=? AND password=?"
13
14 cursor.execute(query, (username, hashed_password))
15
16 if cursor.fetchone():
17     print("Login Successful")
18 else:
19     print("Invalid Credentials")
20
```

Vulnerability	Risk	Solutions
SQL Injection	Unauthorised access	Parameterized queries
Plain Passwords	Password leakage	SHA-256 hashing
NO Validation	Unsafe input	Input sanitization

5. Recommendations

- Always validate user input.
- Use parameterized SQL queries.
- Store passwords securely using hashing.
- Perform regular security testing.

- Avoid exposing sensitive information.

6. Conclusion

This project successfully identified vulnerabilities in a Python login system and implemented secure coding practices to improve application security. The review demonstrated the importance of secure coding in protecting software from cyber threats.